QB: Chapter 2.3 — Queues

Unit 2 | 4 × 2-mark | 5 × 5-mark | **Total: 9 questions** Lectures: L27–L29 | BT Levels: BT1, BT2, BT3 | COs: CO2, CO4

Section A: 2-Mark Questions

Q1. [BT1 | CO2] Define a *queue*. State the FIFO principle. Name the operation used to add an element and the operation used to remove one. What is the *front* and what is the *rear* of a queue?

Q2. [BT1 | CO2] Distinguish between a *queue* and a *deque* (double-ended queue). State which ends of a deque support insertion and deletion. Name one use case that specifically requires a deque.

Q3. [BT2 | CO2] Explain why a naïve array-based queue suffers from *front wastage*. Describe in two sentences how a circular queue with modulo arithmetic resolves this problem. What condition distinguishes a full circular queue from an empty one?

Q4. [BT2 | CO2, CO4] Describe a *min-priority queue* and a *max-priority queue*. What rule governs the order in which elements are removed? Give one real-world application of each type.

Section B: 5-Mark Questions

Q5. [BT3 | CO2] Implement a circular queue with capacity = 5. Trace the following sequence of operations: enqueue(10), enqueue(20), enqueue(30), dequeue(), enqueue(40), enqueue(50), enqueue(60).

After each operation, show: the array contents, the values of `front`, `rear`, and `size`. Clearly handle the overflow attempt and identify when it occurs.

Q6. [BT3 | CO2] Implement a queue using **two stacks** (stack_in and stack_out). Describe the logic for `enqueue` and `dequeue`. Explain the amortised time complexity of dequeue.

Trace the following sequence: enqueue(1), enqueue(2), enqueue(3), dequeue(), dequeue(). After each operation, show the contents of both stacks. Verify the output order follows FIFO.

Q7. [BT3 | CO2] Implement a double-ended queue (deque) using a circular array (capacity = 6). Trace the following operations: insertFront(5), insertRear(10), insertRear(15), deleteFront(), insertFront(3), insertRear(20).

Show the array state, `front`, `rear`, and `size` after each operation.

Q8. [BT3 | CO2] Implement a queue using a singly linked list with `head` (front) and `tail` (rear) pointers. Implement `enqueue(item)` and `dequeue()`.

(a) Explain why `enqueue` uses the `tail` pointer and `dequeue` uses the `head` pointer to achieve O(1) time for both. (b) Trace enqueue(A), enqueue(B), enqueue(C), dequeue(). After each operation, draw the linked list with head and tail pointer positions.

Q9. [BT3 | CO2, CO4] Implement a min-priority queue using Python's `heapq` module. Each entry is a tuple `(priority, task_name)`.

Trace the following insertions: (3, "print"), (1, "emergency"), (2, "backup"), followed by extracting all tasks one by one.

(a) After each insertion, show the internal heap array state. Explain how the min-heap sift-up maintains the heap property. (b) Show the array state after each extraction. Verify that tasks are extracted in ascending priority order. (c) State the time complexity of insertion and extraction in a binary-heap-based priority queue.